



Writing tests

Introduction

Playwright tests are simple: they **perform actions** and **assert the state** against expectations.

Playwright automatically waits for **actionability** checks to pass before performing each action. You don't need to add manual waits or deal with race conditions. Playwright assertions are designed to describe expectations that will eventually be met, eliminating flaky timeouts and racy checks.

You will learn

- [How to write the first test](#)
- [How to perform actions](#)
- [How to use assertions](#)
- [How tests run in isolation](#)
- [How to use test hooks](#)

First test

Take a look at the following example to see how to write a test.

tests/example.spec.ts

```
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  // Expect a title "to contain" a substring.
  await expect(page).toHaveTitle(/Playwright/);
});

test('get started link', async ({ page }) => {
```



```
await page.goto('https://playwright.dev/');

// Click the get started link.
await page.getByRole('link', { name: 'Get started' }).click();

// Expects page to have a heading with the name of Installation.
await expect(page.getByRole('heading', { name: 'Installation'
})).toBeVisible();
});
```

NOTE

Add `// @ts-check` at the start of each test file when using JavaScript in VS Code to get automatic type checking.

Actions

Navigation

Most tests start by navigating to a URL. After that, the test interacts with page elements.

```
await page.goto('https://playwright.dev/');
```

Playwright waits for the page to reach the load state before continuing. Learn more about [page.goto\(\)](#) options.

Interactions

Performing actions starts with locating elements. Playwright uses [Locators API](#) for that. Locators represent a way to find element(s) on the page at any moment. Learn more about the [different types](#) of locators available.

Playwright waits for the element to be [actionable](#) before performing the action, so you don't need to wait for it to become available.

```
// Create a Locator.  
const getStarted = page.getByRole('link', { name: 'Get started' });  
  
// Click it.  
await getStarted.click();
```

In most cases, it'll be written in one line:

```
await page.getByRole('link', { name: 'Get started' }).click();
```

Basic actions

Here are the most popular Playwright actions. For the complete list, check the [Locator API](#) section.

Action	Description
<code>locator.check()</code>	Check the input checkbox
<code>locator.click()</code>	Click the element
<code>locator.uncheck()</code>	Uncheck the input checkbox
<code>locator.hover()</code>	Hover mouse over the element
<code>locator.fill()</code>	Fill the form field, input text
<code>locator.focus()</code>	Focus the element
<code>locator.press()</code>	Press single key
<code>locator.setInputFiles()</code>	Pick files to upload
<code>locator.selectOption()</code>	Select option in the drop down

Assertions

Playwright includes **test assertions** in the form of `expect` function. To make an assertion, call `expect(value)` and choose a matcher that reflects the expectation.

Playwright includes async matchers that wait until the expected condition is met. Using these matchers makes tests non-flaky and resilient. For example, this code waits until the page gets the title containing "Playwright":

```
await expect(page).toHaveTitle(/Playwright/);
```

Here are the most popular async assertions. For the complete list, see [assertions guide](#):

Assertion	Description
<code>expect(locator).toBeChecked()</code>	Checkbox is checked
<code>expect(locator).toBeEnabled()</code>	Control is enabled
<code>expect(locator).toBeVisible()</code>	Element is visible
<code>expect(locator).toContainText()</code>	Element contains text
<code>expect(locator).toHaveAttribute()</code>	Element has attribute
<code>expect(locator).toHaveLength()</code>	List of elements has given length
<code>expect(locator).toHaveText()</code>	Element matches text
<code>expect(locator).toHaveValue()</code>	Input element has value
<code>expect(page).toHaveTitle()</code>	Page has title
<code>expect(page).toHaveURL()</code>	Page has URL

Playwright also includes generic matchers like `toEqual`, `toContain`, `toBeTruthy` that can be used to assert any conditions. These assertions do not use the `await` keyword as they perform immediate synchronous checks on already available values.

```
expect(success).toBeTruthy();
```

Test Isolation

Playwright Test is based on the concept of **test fixtures** such as the **built in page fixture**, which is passed into your test. Pages are **isolated between tests due to the Browser Context**, which is equivalent to a brand new browser profile. Every test gets a fresh environment, even when multiple tests run in a single browser.

tests/example.spec.ts

```
import { test } from '@playwright/test';

test('example test', async ({ page }) => {
  // "page" belongs to an isolated BrowserContext, created for this specific
  test.
});

test('another test', async ({ page }) => {
  // "page" in this second test is completely isolated from the first test.
});
```

Using Test Hooks

You can use various **test hooks** such as `test.describe` to declare a group of tests and `test.beforeEach` and `test.afterEach` which are executed before/after each test. Other hooks include the `test.beforeAll` and `test.afterAll` which are executed once per worker before/after all tests.

tests/example.spec.ts

```
import { test, expect } from '@playwright/test';

test.describe('navigation', () => {
  test.beforeEach(async ({ page }) => {
    // Go to the starting url before each test.
    await page.goto('https://playwright.dev/');
  });

  test('main navigation', async ({ page }) => {
    // Assertions use the expect API.
    await expect(page).toHaveURL('https://playwright.dev/');
  });
});
```

What's Next

- Run single test, multiple tests, headed mode
- Generate tests with Codegen
- See a trace of your tests
- Explore UI Mode
- Run tests on CI with GitHub Actions